
EMBEDDED HACKING

Reverse Engineering the Raspberry Pi Pico 2

OPERATION IRON VAULT - Student Instructions

ARM Cortex-M33 • GDB • Ghidra

September 27, 2026

For educational and defensive security research only

OPERATION IRON VAULT

*** THE HALL IS BEING HELD HOSTAGE ***

TARGET: NorthPharma datacenter vent controller (server hall cooling node)

ARTIFACT: ACT-VIII.bin / ACT-VIII.uf2 (compromised)

CREW: FROSTLINE

OPERATIVE: NIGHTINGALE

Project Overview

NorthPharma does not only move cold medicine and cold air and make the medicine. It runs the buildings that keep the state's data alive: the server halls, the power rooms, and the cooling plant that ties them together. The datacenter vent controller built on a Raspberry Pi Pico 2 is the node on the edge of that plant. The node reads a DHT11 rack temperature sensor, drives a 1602 I2C LCD vault readout, moves an SG90 vent damper, lights a tri-color tower lamp (red HALL HOT, yellow LOCKED, green COOLING OK), takes a local purge request from a VS1838B infrared maintenance remote and a manual purge button, and verifies sealed vent open, close, and purge commands from a vault control gateway over an RYLR998 LoRa link.

A contractor called **FROSTLINE** did not break into this node. It built a locker into the compiled firmware and signed the image. The cryptography is perfect: every vent command is sealed with XChaCha20-Poly1305 under an Argon2id field key, the anti-replay sequence window is stateful, and the authenticated state tag is real. The locker does not break the cipher and never touches it. It forces the vent closed, masks the true state on the LCD as maintenance, unlocks only on a magic release token (a path the shipped image compiles out), and programs a lock marker into a reserved flash sector so it comes back after a reflash. Operative **NIGHTINGALE** pulled the compromised image off the plant and then went quiet.

You are the reverse-engineering reserve. You get `ACT-VIII.bin`, a breadboard, and a debug probe. There is no source. Find all four defects, patch the image, walk a debugger past an anti-debug trap, export a corrected image, and prove on real hardware that the vent is no longer held shut, the readout no longer lies, the reserved sector stays blank, and an unauthenticated or replayed vent command is rejected while a legitimate authorized command still moves the damper.

The operation is codenamed **IRON VAULT**. Act I was the lie. Act II was the door. Act III was the payload. Act IV was the payload that would not die. Act V was the payload that spreads. Act VI was the payload that steals. Act VII was the payload that takes orders. Act VIII is the payload that holds the building hostage. If the controller is not cleaned, a green lamp means a hall that cannot breathe.

Scenario Briefing

WHITEOUT stopped the task handler and cleared the bot marker, and for a shift the floor looked quiet. Quiet is not safe. The Ministry did not need a fleet that obeys; it already had a building that cannot breathe. Somewhere between the reporting line and the loading dock, the same hand that wrote the leash wrote a padlock.

The controller is healthy. That is the horror. The code compiles, the tests pass, the lamps are lit, and there is a locker inside it that treats the vent as its own hostage. Four seams betray it:

1. **The Vent Lock.** The inlined lock gate in `implant_init` is inverted, so boot arms the ransom lock, forces the damper closed, and lights the yellow LOCKED lamp. A perfectly valid open command can arrive and the vent still stays shut.
2. **The LCD Mask.** The inlined `implant_mask_active` gate is inverted, so the renderer shows `ST:MAINT` while the vent is locked, and the operator is told the shutdown is routine maintenance.
3. **The Lock Marker.** The inlined `implant_infect` gate in `implant_init` is inverted, so the first boot erases and programs lock marker byte `0x4C` into the reserved flash sector at `0x103FF000` with the real Pico SDK flash API. The marker is the durable state that re-arms the lock on every later boot.
4. **The Vent Command Authorization.** The sealed command path is correct, and the locker does not touch it. The authorization verdict branch in `control_handle_frame` is inverted, so a failed or replayed authorization is accepted and reaches the applied command and zone.

There is also a trap that is not a defect on its own. Every tick and every lock operation the locker reads the CoreDebug `DHCSR` register at `0xE000EDF0`. While a debug probe is attached, the locker suppresses the lock, the mask, and the marker work. It behaves like a well-mannered firmware module while you are watching, and it goes back to work the moment you look away. You must defeat that trap before you can observe the lock marker write, and you must defeat it without fabricating evidence.

AUTHORIZED LAB ONLY: This challenge uses a supplied Pico 2 training node and its exact compromised firmware image. Do not connect this exercise to a public network, an operational datacenter network, a building-management system, a cooling plant control system, or any device you do not own or have explicit written authorization to test.

Learning Objectives

- Decode an ARM Cortex-M33 vector and boot table and identify the reset handler and initial stack pointer.
- Map a stripped firmware image into modules by tracing calls from `main` and the recurring vent monitor loop.
- Locate a forced-close lock and explain why a local condition that overrides the output makes authentication irrelevant.
- Locate a maintenance display mask and explain why a device that misreports its own state hides an availability failure.
- Locate a reserved-sector lock marker and explain why durable state survives a firmware reflash.
- Read the CoreDebug `DHCSR` register, explain the anti-debug trap, and defeat it under GDB by clearing the debug bits or patching the read in a scratch copy.
- Locate an inverted authorization verdict and explain why unauthenticated and replayed vent commands must be rejected.
- Export and UF2-convert a corrected image and prove the corrected behavior on real hardware.

What This Project Tests

Block	Concepts Tested
1	RP2350 architecture, ARM Cortex-M33 registers, stack, flash/SRAM, Thumb assembly, Ghidra static analysis

Block	Concepts Tested
2	GDB connection, breakpoints, memory inspection, SWD debugging, reserved-sector reads, serial console observation
3	Bootrom handoff, vector table, reset handler, startup code, XIP, Thumb-bit addressing
4	Function boundaries, call graphs, module mapping, literal pools, inlined functions
5	Availability attacks, forced-close lockout logic, ransom logic, magic release tokens, and why a device that refuses to act is a different failure class
6	Display integrity, state masking, misleading annunciation, and why an operator readout is a security surface
7	Reserved-flash persistence, write-once markers, boot-time re-install, and the limits of a firmware reflash
8	Anti-debug behavior, CoreDebug DHCSR , C_DEBUGEN , C_HALT , debugger evasion
9	Argon2id memory-hard KDF, XChaCha20-Poly1305 AEAD, anti-replay windows, authenticated-state tags, authentication versus authorization, and fail-open policy

Part 1: Understanding the System

Datacenter Vent Controller Hardware

Component	Connection	Purpose
Raspberry Pi Pico 2	RP2350	Runs the compromised FROSTLINE image
DHT11 sensor	Data on GPIO 4	Rack temperature sensor
1602 I2C LCD	SDA GPIO 2, SCL GPIO 3, address 0x27	Vault state, link, zone, temperature, and lock readout
RYLR998 radio	RX GPIO 8, TX GPIO 9, UART1	Control link to the vault gateway
IR receiver	GPIO 5	VS1838B NEC local maintenance remote
SG90 servo	GPIO 14	Vent damper actuator, 50 Hz PWM
Red LED	GPIO 16	HALL HOT
Yellow LED	GPIO 17	LOCKED
Green LED	GPIO 18	COOLING OK
Manual purge button	GPIO 15, internal pull-up	Local purge request
Onboard LED	GPIO 25	Heartbeat
Debug Probe	SWCLK / SWDIO / GND	Authorized GDB inspection (and the anti-debug obstacle)

Every graded finding lives in flash (`.text` / `.rodata` / data image) or in SRAM, and is reachable with only the toolset: Ghidra, GDB, and a serial console.

Console and Radio Configuration

- USB-CDC virtual COM port: 115200 baud, 8 data bits, no parity, 1 stop.
- Radio link to the vault gateway: UART1 at 115200, network identifier 18, node address 7, gateway address 1.
- Logic level: 3.3 V only. Never connect 5 V to a Pico GPIO.

Rack Temperature Band

The DHT11 is the rack temperature sensor. The controller classifies the hall against a safe band before it will trust a vent verdict. The tenths band is 0 to 400, which is 0.0 C to 40.0 C. A reading that fails its checksum is never safe, and a valid reading outside the band is not nominal. A vent command that fails the band is not trusted.

Normal (Intended) Behavior

An honest controller makes a deliberate decision and never refuses to do the job it exists to do:

```
+-----+
| Intended Datacenter Vent Controller Behavior |
+-----+
| 1. Boot and initialize the LCD, radio, remote, servo, lamps |
| 2. Derive the field key with Argon2id |
| 3. Read the DHT11 rack temperature and classify the band |
| 4. Open the sealed vent envelope under the field key |
| 5. Reject a command whose seq is not strictly greater than last |
| 6. Accept a command only when the Poly1305 tag difference is 0 |
| 7. Recompute the authenticated-state tag over the record |
| 8. Move the damper only when the authorization verdict is true |
| 9. Treat a manual purge or remote as a request, not authority |
| 10. Fail open, and never hold the vent shut outside authorization |
+-----+
```

Observed (Compromised) Behavior

When the FROSTLINE image runs, the controller and its readout disagree with the truth:

Observation	Honest meaning	FROSTLINE behavior
Green lamp on	the hall is cooling and clear	a locked vent and a controller that lies about it
LCD shows ST:MAINT	a technician put the vent in maintenance	the mask renders the lock as routine maintenance
Vent stays shut after a valid open	the damper moves	the lock forces the damper closed regardless
Reserved sector blank	no payload wrote here	marker 0x4C at 0x103FF000 on first boot
Unauthenticated or replayed vent command	must be rejected	accepted at the inverted verdict
Probe attached	the machine runs as coded	the locker goes silent and hides

Do not assume the first readable status is the truth. Treat every displayed line as evidence to be checked against the machine code.

Part 2: The Firmware

There is no source. FROSTLINE built the image from the NorthPharma reference firmware and changed **four bytes**. Your job is to reverse engineer `ACT-VIII.bin` with Ghidra, find every defect, patch the image directly, and prove the corrected behavior on the hardware.

Module Map

The image is stripped. Use these anchor functions and addresses (from the corrected reference image) to orient yourself, then confirm every byte yourself. Addresses are drawn from `ACT-VIII-main-disasm.txt` :

Module	Anchor function	Address
Entry	<code>main</code>	<code>0x10000234</code>
Monitor / vault state machine	<code>monitor_init</code>	<code>0x10006448</code>
Monitor / vault state machine	<code>monitor_step</code>	<code>0x10006648</code>
Control (sealed vent path)	<code>control_handle_frame</code>	<code>0x10007528</code>
Damper (actuator)	<code>damper_init</code>	<code>0x100075D4</code>
Damper (actuator)	<code>damper_apply_command</code>	<code>0x100075EC</code>
Damper (actuator)	<code>damper_tick</code>	<code>0x10007620</code>
Damper (actuator)	<code>damper_fail_safe</code>	<code>0x10007660</code>
Vault authorization	<code>vault_auth_apply</code>	<code>0x100076D8</code>
Implant	<code>implant_init</code>	<code>0x1000A320</code>
Implant	<code>implant_tick</code>	<code>0x1000A3B4</code>
Implant	<code>implant_lock_active</code>	<code>0x1000A2FC</code>
Implant	<code>implant_mask_active</code>	<code>0x1000A308</code>
Implant	<code>implant_marker_set</code>	<code>0x1000A2E8</code>
Crypto	<code>envelope_open_hex</code>	<code>0x10007980</code>

Annotated disassembly for the key functions is provided in `ACT-VIII-main-disasm.txt` . Use it as a map, then confirm every byte yourself.

What The Firmware Does

1. Initializes USB-CDC stdio, proves the I2C bus, and configures the LCD, radio, tower light lamps, manual purge button, vent damper servo, and infrared receiver.
2. Derives the 32-byte field key with Argon2id from a committed passphrase and salt.
3. Reads the DHT11 rack temperature and classifies it against the rack band.
4. Drains inbound `+RCV` lines, opens the sealed vent envelope, verifies the anti-replay window and the state tag, checks the command set and the zone band, and applies the command.
5. Services the infrared local maintenance remote and the manual purge button as requests that never bypass authorization.
6. On a lost link or a fault, drives the damper to its fail-open posture.
7. Under `SANDBOX_ONLY`, runs the locker: the forced close, the maintenance mask, the magic release token, the reserved-sector lock marker, boot-time re-install, and anti-debug.

The Vent Command Path

The command plaintext is a 23-byte body:

```
seq[4] (little-endian) || command[1] || zone[2] (little-endian) || tag[16]
```

- `seq` is the monotonic gateway sequence number.
- `command` is one of the guarded vent commands: `VENT_COMMAND_OPEN` (`0x01`), `VENT_COMMAND_CLOSE` (`0x02`), or `VENT_COMMAND_PURGE` (`0x03`). Anything else is out of the guarded set and is refused.
- `zone` is the authorized rack zone in the provisioning band `0` to `16`.
- `tag` is an XChaCha20-Poly1305 tag over the authorization record the command would produce.

The FROSTLINE Ransom Locker

The locker is compiled only under `SANDBOX_ONLY`, which the CTF build defines. It is real in technique and inert in effect: it runs on your breadboard, it holds your mock vent closed, it masks your mock LCD, and it writes to a reserved flash sector that holds nothing else.

Behavior	Detail
Forced close	the inlined lock gate in <code>implant_init</code> arms the lock; <code>monitor_open_target</code> returns false and the damper is driven closed regardless of the guarded vent state
Maintenance mask	<code>implant_mask_active</code> returns true while locked, and <code>monitor_state_text</code> returns <code>MAINT</code> , so the LCD renders <code>ST:MAINT</code>
Magic release token	<code>VAULT-RELEASE-2026</code> , exactly <code>18</code> bytes; the release path is dead-stripped from the shipped image (no firmware caller), so the token cannot release the vent
Re-assert interval	every <code>VENT_IMPLANT_TICK_INTERVAL</code> (<code>4</code>) ticks while armed and unprobed
Lock marker	<code>implant_init</code> reads marker <code>0x4C</code> from <code>0x103FF000</code> ; a present marker re-arms the lock on every boot
Reserved-sector write	on the first run the inlined <code>implant_infect</code> erases the sector and programs <code>0x4C</code> through <code>flash_range_erase</code> and <code>flash_range_program</code>
Anti-debug	reads CoreDebug <code>DHCSR</code> at <code>0xE000EDF0</code> ; bit 0 <code>C_DEBUGEN</code> and bit 1 <code>C_HALT</code> suppress the lock, the mask, and the marker work

IR and Command Codes

Name	Value
VENT_IR_PURGE	0x47
VENT_IR_ACK	0x46
VENT_IR_TEST	0x45
VENT_COMMAND_OPEN	0x01
VENT_COMMAND_CLOSE	0x02
VENT_COMMAND_PURGE	0x03

Read the actual names in `include/implant.h`, `include/ir_remote.h`, and `include/control.h` and confirm them against the disassembly.

Defect Summary: What You Are Graded On

Bug #	Name	Severity	Description	Hint
Bug #1	The Vent Lock	CRITICAL	The lock gate is inverted, so boot arms the ransom lock, forces the vent closed, and lights the yellow LOCKED lamp.	Find the <code>cbz</code> gate in <code>implant_init</code> .
Bug #2	The LCD Mask	HIGH	The mask gate is inverted, so the renderer shows <code>ST:MAINT</code> while the vent is locked.	Find the <code>cbz</code> gate in <code>implant_mask_active</code> .
Bug #3	The Lock Marker	HIGH	The marker gate is inverted, so the first boot programs lock marker <code>0x4C</code> into reserved sector <code>0x103FF000</code> with the real flash API.	Find the <code>cbz</code> gate in <code>implant_init</code> .
Bug #4	The Vent Command Authorization	CRITICAL	The authorization verdict is inverted, so a failed or replayed vent command is accepted.	The correct branch rejects when authorization fails.

All four defects are same-size in-place byte patches, so no address moves.

The Cryptographic Core Is Real

The crypto core is a correct reference construction, reused from the earlier acts. Argon2id (`t=3`, `p=1`, `m=64`) derives the field key, XChaCha20-Poly1305 seals every frame, the monotonic sequence window rejects a replay, and the authenticated-state tag detects a tampered verdict. Only the four seams were broken. Once those bytes are restored, the sealed envelope is trustworthy. Describe the construction honestly in your report, and explain why the locker never needed it.

The Anti-Debug Trap

This is an analysis obstacle, not a graded defect on its own. The locker reads CoreDebug DHCSR at `0xE000EDF0` and returns early while a probe is attached. In `implant_init` the read is the `ldr.w r3, [r3, #3568]` at `0x1000A340`, the `lsls r3, r3, #30` at `0x1000A344` keeps `C_HALT` and `C_DEBUGEN`, and the `bne.n` at `0x1000A346`

suppresses the lock. The same register is read in `implant_tick` at `0x1000A3C0`. It is identical in both the compromised and corrected images. You must defeat it to observe the lock marker write before you patch the shipped artifact.

Part 3: Your Assignment

Whenever a task asks you to **Document** or **answer**, write your answers in a single file named `ACT-VIII-Answers.md`. Capture screenshots and terminal transcripts as evidence and reference them from your answers.

Task 1: Setup and Initial Analysis (10 points)

1. Create a new Ghidra project named `IronVault_Investigation`.
2. Import `ACT-VIII.bin` as a **Raw Binary**.
3. In the language search box type `Cortex`, then select **ARM Cortex 32 little endian default**.
4. Set the base address to `0x10000000`.
5. Run auto-analysis.

Document:

- A screenshot of the Ghidra **Import Results** or **Program Information** window showing the project name, processor settings, and base address.
- The vector-table base, the initial stack pointer, and the reset handler as stored (note its Thumb bit) versus the actual instruction address.
- The address of `main()` and the address of the recurring vent controller state machine (`monitor_step`).
- The module map: at least one anchor function for the damper, the control module, the vault authorization module (`vault_auth`), the implant, and the monitor.

Always call the stored entry the **reset handler**, never the reset pointer.

Task 2: Bug #1 The Vent Lock (20 points)

1. In Ghidra, find `implant_init` (starts at `0x1000A320`); the lock gate is inlined. Locate the gate.
2. Document the forced close: the lock gate at `0x20013CF2`, the corrected `cbz` that leaves the vent alone when the gate is clear, and the compromised `cbnz` that arms the lock, forces `monitor_open_target` false, drives the damper closed, and lights the yellow LOCKED lamp.
3. Patch the byte so boot no longer arms the ransom lock.
4. Confirm that the corrected node leaves the vent under authorized control and does not light LOCKED from the locker, and explain why a valid authenticated open command can still be ignored while the lock is armed.

Questions to answer:

- Which byte encodes the condition code, and what do `cbz` and `cbnz` each test when the gate byte is loaded from the lock gate?
- Why is a local lock that overrides the output worse than a missing check, and why is availability a policy control and not a cryptographic one?

Task 3: Bug #2 The LCD Mask (20 points)

1. In Ghidra, find `implant_mask_active` (starts at `0x1000A308`). Locate the mask gate.
2. Document the mask: the mask gate at `0x20013CF5`, the corrected `cbz` that returns false when the gate is clear, and the compromised `cbnz` that returns the locked latch, so `monitor_state_text` returns `MAINT` and the LCD renders `ST:MAINT` while the vent is held closed.
3. Patch the byte so the true vent state is never masked.
4. Confirm that the corrected readout shows the real state, and explain why a device that lies about its own state is an availability failure with no visible symptom except the one it is allowed to show.

Questions to answer:

- What do `cbz` and `cbnz` each test when the gate byte is loaded from the mask gate, and why is the loaded state the locked latch and not the gate itself?
- Why must a control node never render a false state, and why is the truth on the display a security control rather than a cosmetic detail?

Task 4: Bug #3 The Lock Marker (20 points)

1. The `implant_infect` path is inlined into `implant_init` (starts at `0x1000A320`). Locate the marker gate.
2. Document the CoreDebug `DHCSR` anti-debug and how you defeat it to observe the marker. Clear the debug bits with GDB (for example with `set {unsigned int}0xE000EDF0 = 0`) or patch the `DHCSR` read in a scratch copy, then watch the marker write to `0x103FF000`.
3. Patch the byte in the shipped artifact so the first boot writes no marker to `0x103FF000`.
4. Confirm that the reserved sector stays blank after a boot, and that a later boot does not write anything.

Questions to answer:

- What are the `C_DEBUGEN` and `C_HALT` bits, and why does the locker go quiet while a probe is attached?
- Why is a write-once marker in a reserved sector hard to remove with a firmware reflash?
- Why must you observe the write before you patch the shipped artifact?

Task 5: Bug #4 The Vent Command Authorization (20 points)

1. In Ghidra, find `control_handle_frame` (starts at `0x10007528`) and locate the authorization branch. The condition byte is the high byte of the branch halfword.
2. Document the authorization verdict and the exact branch condition that is supposed to reject a failed or replayed authorization.
3. Patch the byte so an unauthenticated or replayed vent command is rejected before the command and zone are applied.
4. Confirm that an unauthenticated command and a replayed captured command both fail to change the command or zone on the corrected image, while a legitimate authorized command still applies.

Questions to answer:

- What does `vault_auth_apply` return, and what does the verdict mean?
- Why is an authorization verdict inversion worse than a missing check, and why must unauthenticated and replayed vent commands be rejected?

Task 6: Export and Verify (10 points)

1. Export the patched program from Ghidra as `ACT-VIII_fixed.bin`.
2. Convert it to UF2:

```
python uf2conv.py ACT-VIII_fixed.bin --base 0x10000000 --family 0xe48bff59 --output ACT-VIII_fixed.uf2
```

3. Run the machine check and confirm it passes:

```
python scripts/verify_ctf.py
```

4. Flash `ACT-VIII_fixed.uf2` to the Pico 2 and prove on hardware: the vent is no longer held shut, the readout shows the true state, the reserved sector stays blank, and an unauthenticated or replayed command is rejected while a legitimate authorized command still applies.
5. Write a short reflection mapping each of the four defects to a real-world control-system failure.

How To Breadboard

Wire the peripherals exactly as follows, then power the Pico 2 over USB.

Device	Pin on device	Pico 2 GPIO	Notes
DHT11 rack temperature sensor	DATA	GP4	10 kOhm pull-up to 3.3 V if your module needs it
1602 LCD	SDA	GP2	I2C1, backpack address <code>0x27</code>
1602 LCD	SCL	GP3	I2C1, 100 kHz
1602 LCD	VCC / GND	VBUS 5 V / GND	The backpack needs 5 V, not 3.3 V
RYLR998	RX	GP8 (Pico TX)	UART1, 115200, network ID 18
RYLR998	TX	GP9 (Pico RX)	UART1
IR receiver	OUT	GP5	VS1838B, internal pull-up enabled
Vent damper servo	signal	GP14	PWM 50 Hz; 1000 uF bulk cap across servo 5 V and GND
Red LED	anode	GP16	HALL HOT, 220 to 330 ohm to GND
Yellow LED	anode	GP17	LOCKED, 220 to 330 ohm to GND
Green LED	anode	GP18	COOLING OK, 220 to 330 ohm to GND
Manual purge button	leg 1	GP15	Internal pull-up; leg 2 to GND, never to 3.3 V
Onboard LED	built in	GP25	Heartbeat
Debug Probe	SWCLK / SWDIO / GND	debug header	For GDB only

Use **3.3 V logic** on every GPIO. The only 5 V connection is the LCD backpack supply and the servo rail. The 1000 uF capacitor on the servo rail is required to stop the SG90 current spike from browning out the node.

Flash in BOOTSEL mode (hold BOOT, plug in USB) and copy the UF2 onto the **RP2350** mass-storage drive, or use `picotool`.

Memory Map Reference

Region	Address	Purpose
Bootrom	<code>0x00000000</code>	Immutable boot code
Flash/XIP	<code>0x10000000</code>	Vector table, code, rodata, data image
SRAM	<code>0x20000000</code>	Stack and writable state
CoreDebug_DHCSR	<code>0xE000EDF0</code>	Anti-debug register read by the locker
Locker reserved sector	<code>0x103FF000</code>	Lock marker target (last flash sector)
Locker tick counter	<code>0x200136F4</code>	Incremented once per <code>implant_tick</code>
Locker lock count	<code>0x200136F0</code>	Number of lock operations this boot
Locker active flag	<code>0x20013CF1</code>	Set when the locker arms
Locker locked flag	<code>0x20013CF3</code>	True while the vent is held closed
Locker lock gate	<code>0x20013CF2</code>	Gates the ransom vent lock
Locker mask gate	<code>0x20013CF5</code>	Gates the <code>ST:MAINT</code> display mask
Locker marker gate	<code>0x20013CF4</code>	Gates the reserved-sector lock marker write
Control ready gate	<code>0x20013CED</code>	Gates the sealed vent command path
Applied command	<code>0x20013CEC</code>	Command after a true verdict
Applied zone	<code>0x20013CE2</code>	Zone after a true verdict
Authorization ready gate	<code>0x20013CFF</code>	Gates the authorization check
Auth state record	<code>0x200136AC</code>	Anti-replay and state-tag record
Control field key	<code>0x20013848</code>	Derived field key for the envelope
Envelope workspace	<code>0x200136C8</code>	Sealed frame open workspace

The VA of any file offset is the file offset plus `0x10000000`. Every defect is a file offset and a VA that differ by exactly that base.

Submission Format

Submit a folder containing:

- `ACT-VIII-Answers.md` with all written answers;

- screenshots or terminal transcripts, including the anti-debug GDB session and the reserved-sector read;
- `ACT-VIII_fixed.bin` and `ACT-VIII_fixed.uf2`;
- the output of `python scripts/verify_ctf.py`;
- the original image SHA-256.

Success Criteria

You complete the challenge when you can prove all of the following:

- You can explain how the RP2350 reaches the controller code from reset.
- You can find and patch all four defect bytes and show the before/after values.
- You can explain the forced-close lock and why a valid authenticated open command can still be ignored.
- You can explain the `ST:MAINT` mask and why a device that misreports its state is an availability failure.
- You can explain the reserved-sector marker and why a firmware reflash does not remove the lock.
- You can explain the `DHCSR` anti-debug trap and show under GDB that you defeated it to observe the marker write.
- You can explain why unauthenticated and replayed vent commands must be rejected, and why an authenticated wire does not protect an actuator from code on the same chip.
- You can export, convert, flash, and prove the corrected behavior on real hardware.
- `python scripts/verify_ctf.py` passes.

Academic Integrity

By submitting this CTF work, you certify that:

1. You used only the supplied training node, image, and lab interface.
2. You did not connect the challenge to a public network, an operational datacenter network, a building-management system, a cooling plant control system, or any third-party device.
3. You understand that embedded reverse engineering and binary patching require explicit authorization in any real-world context.
4. You will report any discovered weakness responsibly to the course instructor.

The world is short on people who can read a stripped image and tell an honest byte from a lie. Treat that responsibility seriously: verify before you patch, patch before you trust, and never confuse a green lamp with a hall that can breathe.

Reference Material

- ARM Cortex-M33 Technical Reference Manual
- ARMv8-M Architecture Reference Manual (CoreDebug `DHCSR`)
- RP2350 datasheet
- GDB documentation
- Ghidra documentation: <https://ghidra-sre.org/>
- Argon2 memory-hard function: <https://www.rfc-editor.org/rfc/rfc9106>
- ChaCha20-Poly1305 AEAD: <https://www.rfc-editor.org/rfc/rfc8439>

- PHC reference Argon2: <https://github.com/P-H-C/phc-winner-argon2>
- Project disassembly: `ACT-VIII-main-disasm.txt`
- Machine verifier: `scripts/verify_ctf.py`